

Deep Learning Cheatsheet

1. Key Python Modules

Module	Purpose
TensorFlow	Open-source deep learning framework by Google.
PyTorch	Open-source deep learning framework by Facebook.
Keras	High-level API for TensorFlow (e.g., <code>tf.keras</code>).
NumPy	Numerical computing in Python (e.g., arrays, matrices).
Pandas	Data manipulation and analysis.
Matplotlib	Plotting and visualization.
Scikit-learn	Machine learning tools (e.g., preprocessing, evaluation).
Transformers	Pre-trained NLP models (e.g., BERT, GPT).
OpenCV	Computer vision tasks (e.g., image processing).
Torchvision	Datasets and models for computer vision in PyTorch.
Datasets	Hugging Face library for datasets.

2. Neural Network Architectures

Feedforward Neural Network (FNN)

python

```
import tensorflow as tf
from tensorflow.keras import layers
```

```
model = tf.keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(784,)),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

Diagram:

Input -> Dense -> ReLU -> Dense -> ReLU -> Dense -> Softmax -> Output

Use Case: Classification, regression.

Convolutional Neural Network (CNN)

python

```
model = tf.keras.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

Diagram:

Input -> Conv2D -> ReLU -> MaxPooling -> Conv2D -> ReLU -> Flatten -> Dense -> Softmax -> Output

Use Case: Image classification, object detection.

Recurrent Neural Network (RNN)

python

```
model = tf.keras.Sequential([
    layers.SimpleRNN(64, return_sequences=True, input_shape=(10, 28)),
    layers.SimpleRNN(64),
    layers.Dense(10, activation='softmax')
])
```

Diagram:

Input -> RNN -> RNN -> Dense -> Softmax -> Output

Use Case: Time series, text generation.

Transformer

python

```
from transformers import BertTokenizer, BertForSequenceClassification
```

```
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertForSequenceClassification.from_pretrained('bert-base-uncased')
```

Diagram:

Input -> Tokenizer -> Transformer Encoder -> Classification Head -> Output

Use Case: NLP tasks (e.g., sentiment analysis).

3. Loss Functions

Loss Function

Binary Crossentropy	Binary classification.
Categorical Crossentropy	Multi-class classification.
Mean Squared Error (MSE)	Regression.
Huber Loss	Robust regression.
Kullback-Leibler Divergence (KL)	Variational autoencoders.

Use Case

4. Optimizers

OptimizerDescription

SGD	Stochastic Gradient Descent.
Adam	Adaptive Moment Estimation (most popular).
RMSprop	Root Mean Square Propagation.
Adagrad	Adaptive Gradient Algorithm.

5. Activation Functions

FunctionDescription

ReLU	Rectified Linear Unit (default choice).
Sigmoid	For binary classification.
Softmax	For multi-class classification.
Tanh	Hyperbolic tangent (output in [-1, 1]).

6. Regularization Techniques

Technique

Description

Dropout	Randomly drops neurons during training.
L2 Regularization	Adds penalty for large weights.

Copy

Copy

Copy

Copy

Technique	Description
-----------	-------------

Batch Normalization	Normalizes layer outputs.
---------------------	---------------------------

7. Evaluation Metrics

Metric	Use Case
--------	----------

Accuracy	Classification.
----------	-----------------

Precision/Recall	Imbalanced datasets.
------------------	----------------------

F1 Score	Balance between precision and recall.
----------	---------------------------------------

Mean Absolute Error (MAE)	Regression.
---------------------------	-------------

8. Data Preprocessing

Normalization

```
python
```

```
x_train = x_train / 255.0
```

```
x_test = x_test / 255.0
```

One-Hot Encoding

```
python
```

```
from tensorflow.keras.utils import to_categorical
```

```
y_train = to_categorical(y_train, num_classes=10)
```

Data Augmentation

```
python
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
datagen = ImageDataGenerator(rotation_range=20, width_shift_range=0.2, height_shift_range=0.2)
```

9. Training a Model

```
python
```

```
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

```
history = model.fit(x_train, y_train, epochs=10, validation_data=(x_test, y_test))
```

10. Saving and Loading Models

Saving

```
python
```

```
model.save('my_model.h5')
```

Loading

```
python
```

```
from tensorflow.keras.models import load_model
```

```
model = load_model('my_model.h5')
```

11. Visualization

Training History

```
python
```

```
import matplotlib.pyplot as plt
```

```
plt.plot(history.history['loss'], label='Training Loss')
```

```
plt.plot(history.history['val_loss'], label='Validation Loss')
```

```
plt.legend()
```

```
plt.show()
```

Confusion Matrix

```
python
```

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
```

```
cm = confusion_matrix(y_true, y_pred)
sns.heatmap(cm, annot=True, fmt='d')
```

12. Advanced Techniques

Transfer Learning

```
python
base_model = tf.keras.applications.ResNet50(weights='imagenet', include_top=False)
x = base_model.output
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(1024, activation='relu')(x)
predictions = layers.Dense(10, activation='softmax')(x)
model = tf.keras.Model(inputs=base_model.input, outputs=predictions)
```

Copy

GANs

```
python
# Generator
generator = tf.keras.Sequential([
    layers.Dense(256, activation='relu', input_shape=(100,)),
    layers.Dense(784, activation='tanh')
])
```

Copy

```
# Discriminator
discriminator = tf.keras.Sequential([
    layers.Dense(256, activation='relu', input_shape=(784,)),
    layers.Dense(1, activation='sigmoid')
])
```

13. Diagrams

Neural Network

Input -> Hidden Layer -> Activation -> Output

Copy

CNN

Input -> Conv2D -> ReLU -> MaxPooling -> Flatten -> Dense -> Output

Copy

RNN

Input -> RNN Cell -> RNN Cell -> Dense -> Output

Copy

Transformer

Input -> Tokenizer -> Encoder -> Decoder -> Output

Copy